

École Supérieure Privée d'Ingénierie et de Technologies — ESPRIT

Département Informatique — Filière Data Science & IA

Détection intelligente d'anomalies financières

*Analyse de transactions, modélisation par apprentissage automatique
et génération d'explications par modèle de langage*

Rapport de Projet de Fin d'Études

Réalisé dans le cadre du cycle ingénieur — Spécialité Intelligence Artificielle

Réalisé par : Yousra Chaieb

Encadrant académique : Pr. Mohamed Islem Samaali

Encadrants entreprise : Mr. Slim Zribi & Mme. Hajer

Année universitaire : 2025/2026

Dédicaces

Je dédie ce modeste travail

*À mon père,
source inépuisable de force et de sagesse,
dont le soutien indéfectible m'a porté tout au long de ce chemin.*

*À ma mère,
pour son amour inconditionnel, sa tendresse et ses prières
qui ont illuminé chacune de mes journées.*

*À ma chère sœur **Eya**,
pour sa complicité, son encouragement constant
et les moments de joie partagés.*

*À mes amis,
pour leur présence, leur bonne humeur et leur précieux soutien
tout au long de cette aventure.*

*À toutes les personnes qui m'ont encouragé et aidé
durant mon stage de fin d'études,
et qui ont contribué, de près ou de loin,
à la réussite de ce projet.*

Remerciements

Après avoir rendu grâce à **Dieu**, le Tout-Puissant et le Miséricordieux, qui m'a accordé la volonté, la patience et la santé nécessaires pour mener à bien ce projet de fin d'études.

Je tiens à exprimer ma profonde gratitude à mon encadrant académique, **Mr. Mohamed Islem Samaali**, pour ses conseils avisés, sa disponibilité permanente et ses orientations scientifiques qui ont été déterminantes pour structurer ma démarche et enrichir mes réflexions tout au long de ce travail.

Mes sincères remerciements s'adressent également à mes encadrants en entreprise, **Mr. Slim** et **Madame Hajer**, pour leur accueil chaleureux, la confiance qu'ils m'ont accordée et les précieuses orientations métier qui m'ont permis de mieux appréhender les enjeux réels de la détection d'anomalies financières.

Je remercie tout particulièrement **Madame Emna Zanni**, responsable du département Audit IT, pour son soutien, sa bienveillance et les ressources qu'elle a mises à ma disposition, contribuant ainsi de manière significative à la réussite de ce projet.

Enfin, je remercie chaleureusement mes **camarades de stage** pour les échanges enrichissants, la bonne ambiance et l'entraide qui ont rendu cette expérience encore plus formatrice et agréable.

Résumé

Ce projet de fin d'études porte sur la détection intelligente d'anomalies financières à partir de données transactionnelles réelles collectées auprès d'un client dans le cadre d'une mission d'audit réalisée au sein de **PwC Tunisie**. Conduit selon la méthodologie **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), il vise à construire une chaîne complète et reproductible allant de la compréhension métier et des données jusqu'au déploiement d'un pipeline de détection opérationnel.

Le corpus utilisé comprend environ **200 000 transactions financières**, dont 258 opérations frauduleuses (ratio 1 :774). La sévérité du déséquilibre des classes représente le défi central du projet et justifie l'emploi de stratégies spécifiques telles que la pondération des classes, le rééchantillonnage SMOTE et une évaluation fondée sur le rappel, la précision et le F1-score plutôt que sur la simple exactitude.

Trois approches de modélisation sont comparées : (1) une **régression logistique** servant de baseline linéaire, (2) une **forêt aléatoire** constituant la référence non-linéaire supervisée, et (3) un **autoencoder** profond non supervisé entraîné exclusivement sur les transactions légitimes.

Afin d'améliorer l'interprétabilité, une double couche explicative a été mise en place : les méthodes **SHAP** et **LIME** pour identifier les variables les plus contributives, et une intégration avec **Ollama** pour générer des explications en langage naturel accessibles aux analystes métier.

Mots-clés : détection d'anomalies, fraude financière, CRISP-DM, autoencoder, forêt aléatoire, SMOTE, SHAP, LIME, Ollama, interprétabilité, PwC Tunisie.

Abstract

This final-year project focuses on intelligent anomaly detection in financial transactions using a dataset of real transactional records collected from a client engagement conducted within **PwC Tunisia**. Following the **CRISP-DM** methodology, the goal is to build a complete and reproducible pipeline covering business understanding, data exploration, preparation, modeling, evaluation, and deployment.

The dataset comprises approximately **200,000 financial transactions**, of which only 258 are fraudulent (ratio 1 :774). Three modeling approaches are benchmarked : (1) **logistic regression** as a linear baseline, (2) **random forest** as a robust supervised reference, and (3) a deep **autoencoder** trained exclusively on legitimate transactions. To enhance interpretability, a two-layer explainability stack was built : **SHAP** and **LIME** methods, and an **Ollama** integration to generate natural-language explanations accessible to business analysts.

Keywords : anomaly detection, financial fraud, CRISP-DM, autoencoder, random forest, SMOTE, SHAP, LIME, Ollama, explainability, PwC Tunisia.

Table des matières

Table des figures

Liste des tableaux

Liste des abréviations

AE	AutoEncoder
AUC	Area Under the Curve
BN	Batch Normalization
CRISP-DM	Cross-Industry Standard Process for Data Mining
EDA	Exploratory Data Analysis (Analyse Exploratoire des Données)
ERP	Enterprise Resource Planning (Progiciel de Gestion Intégré)
F1	F1-Score
FN	Faux Négatifs
FP	Faux Positifs
LLM	Large Language Model (Grand Modèle de Langage)
LIME	Local Interpretable Model-agnostic Explanations
LR	Logistic Regression (Régression Logistique)
ML	Machine Learning (Apprentissage Automatique)
MSE	Mean Squared Error (Erreur Quadratique Moyenne)
ONG	Organisation Non Gouvernementale
PR-AUC	Area Under the Precision-Recall Curve
RF	Random Forest (Forêt Aléatoire)
RCM	Risk & Control Matrix (Matrice de Risques et de Contrôles)
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
TP	Vrais Positifs
TN	Vrais Négatifs
VAE	Variational AutoEncoder
XAI	Explainable Artificial Intelligence

Introduction générale

Au cours de mon stage au sein du département Risk Assurance Services de PwC Tunisie, j'ai été confrontée à une réalité concrète : dans le cadre d'une mission d'audit IT, l'équipe dispose de plusieurs centaines de milliers de lignes transactionnelles à analyser, et seule une poignée d'entre elles dissimule une anomalie. Repérer ces cas dans une masse de données aussi volumineuse, sans passer à côté d'une fraude ni submerger les auditeurs de fausses alertes — voilà le défi auquel répond ce projet.

Ce travail propose une **chaîne de traitement complète** pour la détection intelligente d'anomalies financières. Développé selon la méthodologie **CRISP-DM**, il couvre l'intégralité du cycle : de la compréhension des besoins métier à la construction d'un pipeline de déploiement reproductible, en passant par l'analyse exploratoire des données, leur préparation, l'entraînement de plusieurs modèles et leur évaluation rigoureuse.

Deux défis techniques ont structuré l'ensemble de la démarche : (1) la **gestion du déséquilibre extrême** des classes — moins de 0,13 % des transactions sont frauduleuses — et (2) l'**interprétabilité** des décisions, rendue possible grâce aux méthodes SHAP, LIME et à la génération d'explications textuelles par le modèle de langage Ollama.

Organisation du rapport : Ce document est structuré en sept chapitres :

- Le **chapitre 1** présente l'organisme d'accueil PwC Tunisie, le cadre du stage et les objectifs du projet.
- Le **chapitre 2** établit la compréhension métier du problème, la problématique et les critères de succès (phase 1 CRISP-DM).
- Le **chapitre 3** présente le jeu de données transactionnel et l'analyse exploratoire réalisée (phase 2 CRISP-DM).
- Le **chapitre 4** détaille les opérations de préparation des données : nettoyage, ingénierie de variables et gestion du déséquilibre (phase 3 CRISP-DM).
- Le **chapitre 5** décrit les modèles construits : régression logistique, forêt aléatoire et autoencoder (phase 4 CRISP-DM).
- Le **chapitre 6** présente l'évaluation comparative des modèles et les mécanismes d'explicabilité (phase 5 CRISP-DM).
- Le **chapitre 7** aborde le déploiement, l'architecture technique et la reproductibilité du pipeline (phase 6 CRISP-DM).

1 Présentation de l'organisme d'accueil

Introduction

Ce premier chapitre pose le cadre institutionnel et professionnel dans lequel s'inscrit ce projet de fin d'études. Avant de décrire les choix techniques et les résultats obtenus, il est indispensable de comprendre l'environnement au sein duquel la problématique a émergé. Nous présenterons successivement le réseau PwC à l'échelle internationale, son implantation tunisienne, puis le département Risk Assurance Services et sa cellule Audit IT — contexte direct de production des données utilisées dans ce travail. Nous concluons par la description du cadre du stage et des missions confiées.

1.1 PwC à l'échelle internationale

1.1.1 Historique et positionnement

PwC est un réseau mondial de cabinets de services professionnels couvrant l'audit, le conseil, la fiscalité et les transactions. Sa clientèle est très diversifiée : grands groupes internationaux cotés, PME en pleine croissance, institutions publiques et organisations à but non lucratif. Ce qui caractérise le réseau, c'est sa capacité à articuler une expertise internationale uniformisée avec une connaissance fine des réalités locales — une posture particulièrement pertinente sur des marchés comme la Tunisie, où les spécificités réglementaires et économiques sont fortes.

PwC est né en **1998** de la fusion de deux cabinets alors centenaires : *Price Waterhouse*, fondé à Londres en 1849, et *Coopers & Lybrand*, dont les origines remontent à 1854. L'entité fusionnée, d'abord connue sous le nom PricewaterhouseCoopers, a adopté en septembre 2010 son identité actuelle abrégée « PwC ». Aujourd'hui présent dans plus de **187 pays** avec **771 bureaux** et **364 000 collaborateurs**, le réseau se positionne comme le premier cabinet mondial d'audit et de conseil, devant Deloitte, EY et KPMG.

1.1.2 PwC dans le Big Four

PwC s'inscrit dans le cercle restreint des quatre grands cabinets mondiaux d'audit et de conseil — le « Big Four » —, aux côtés de Deloitte, Ernst & Young et KPMG. Chacun de ces réseaux repose sur le même modèle organisationnel : des entités nationales juridiquement indépendantes, fédérées sous une marque commune et soumises à des standards partagés en matière de qualité et d'éthique professionnelle.

TABLE 1.1 – Fiche signalétique de PwC International

Critère	Information
Dénomination	PricewaterhouseCoopers (PwC)
Date de création	1998 (fusion Price Waterhouse & Coopers & Lybrand)
Siège social	Londres, Royaume-Uni
Secteur d'activité	Audit, Conseil, Fiscalité, Transactions
Nombre de pays	Plus de 187
Nombre de bureaux	Plus de 771
Effectif mondial	Plus de 364 000 collaborateurs
Appartenance	Big Four de l'audit

1.1.3 Statut juridique

Sur le plan juridique, PwC s'appuie sur *PricewaterhouseCoopers International Limited* (PwCIL), une société de droit anglais qui joue un rôle de coordination stratégique sans être elle-même prestataire de services. Chaque cabinet national est une entité indépendante, ce qui lui permet de satisfaire aux exigences des pays imposant un actionnariat local pour les cabinets d'audit. En contrepartie de l'usage de la marque PwC, ces entités s'engagent à respecter les standards de qualité, de gestion du risque et d'éthique professionnelle définis par PwCIL.

1.1.4 Stratégie mondiale : « The New Equation »

Pour anticiper les évolutions du marché — notamment l'essor de l'intelligence artificielle et les nouvelles exigences réglementaires —, PwC a structuré sa stratégie globale autour du programme « **The New Equation** », articulé autour de deux axes :

- **Building Trust** : répondre aux attentes en matière de transparence en combinant expertise d'audit, de fiscalité et de cybersécurité, enrichies par l'intelligence artificielle.
- **Sustained Outcomes** : mobiliser l'ensemble des expertises du réseau pour accompagner ses clients vers des résultats durables, couvrant stratégie, services numériques, fiscalité et conformité réglementaire.

1.2 PwC Tunisie

1.2.1 Présentation générale

Fondée en **1983**, PwC Tunisie s'impose comme l'une des premières sociétés tunisiennes dans le domaine de l'audit et du conseil, forte de plus de quarante années d'expérience au service du tissu économique local. Elle propose une gamme étendue de services adaptés au marché tunisien, couvrant aussi bien les besoins des institutions publiques que ceux des entreprises privées, des ONG et des bailleurs de fonds internationaux.

TABLE 1.2 – Fiche signalétique de PwC Tunisie

Critère	Information
Dénomination	PricewaterhouseCoopers Tunisie
Date de création	1983
Siège social	Tunis, Tunisie
Secteur d'activité	Audit, Conseil, Fiscalité, Transactions
Domaines d'intervention	Secteur public, Privé, ONG, Bailleurs de fonds
Appartenance	Réseau PwC International

1.2.2 Offres de services de PwC Tunisie

PwC Tunisie s'organise en quatre lignes de services complémentaires :

Audit. Évaluation du contrôle interne, conception d'organigrammes, développement de manuels de procédures, construction de matrices de risques et de contrôles (Risk & Control Matrix — RCM).

Transactions (Deals). Due diligence acheteur et vendeur, études de faisabilité, élaboration de business plans, évaluation d'entreprises et gestion des processus d'ouverture de capital.

Services Juridiques et Fiscaux. Conseil fiscal, tax due diligence, audit fiscal, tax compliance et assistance lors des contrôles fiscaux.

Consulting. Définition de stratégies de développement, refonte commerciale, amélioration de l'efficacité opérationnelle et conseil en marketing.

1.2.3 Références clients

PwC Tunisie opère dans tous les domaines d'activité, avec une expertise forte dans les services financiers, le secteur public, l'énergie, les projets d'infrastructure et les télécommunications.

1.3 Le département Risk Assurance Services

1.3.1 Présentation générale

Le département **Risk Assurance Services** (anciennement Conseil Audit Formation) est le pôle au sein duquel j'ai réalisé ce stage. Sa mission centrale : aider les organisations clientes à identifier et cartographier leurs risques opérationnels et systémiques, puis à mettre en place les dispositifs de contrôle appropriés. Concrètement, les missions couvrent un spectre large — de l'évaluation du contrôle interne à la revue des systèmes d'information, en passant par l'assistance à la conception de procédures et la formation des équipes audit.

1.3.2 Offres de prestations

Le département apporte une valeur ajoutée à travers trois formes d'intervention complémentaires :

Outsourcing des travaux. Cette prestation s'adresse aux organisations ne disposant pas d'une fonction d'audit interne constituée. PwC prend alors en charge l'intégralité des travaux d'audit, assurant une couverture complète de l'ensemble des missions inscrites au plan d'audit interne.

Co-sourcing des travaux. L'équipe d'audit interne du client fait appel à l'expertise de PwC pour renforcer ses capacités. Cette modalité collaborative favorise le transfert de connaissances et permet à l'équipe interne de monter progressivement en compétences.

Services de conseil. PwC accompagne les entreprises dans l'évaluation de leurs opérations et dans la structuration ou l'amélioration de leur fonction d'audit interne, afin de la rendre plus performante et mieux alignée sur leurs enjeux stratégiques.

1.3.3 La cellule Audit IT

Au cœur du département Risk Assurance Services, la cellule **Audit IT** se consacre à l'examen des systèmes d'information qui alimentent la comptabilité et les états financiers des clients : ERP, systèmes de gestion de trésorerie, plateformes de paiement. Son rôle est de vérifier que ces environnements produisent des données intègres, fiables et conformes aux normes d'audit — ce qui conditionne directement la validité des assertions financières sur lesquelles repose l'opinion du commissaire aux comptes.

C'est précisément dans ce cadre opérationnel que les données de ce projet ont été collectées : un corpus de transactions financières extraites du système d'information d'un client de PwC Tunisie, dans le cadre d'une mission d'audit IT active. Pour des raisons de confidentialité, toutes les informations permettant d'identifier les parties ont été anonymisées.

1.4 Cadre du stage et projet de fin d'études

1.4.1 Objectifs du stage

Ce stage de fin d'études, réalisé au sein du département Risk Assurance Services de PwC Tunisie, a pour objectif principal de concevoir et de déployer un **système de détection intelligente d'anomalies financières** fondé sur des techniques avancées d'apprentissage automatique. Il s'inscrit dans une démarche d'innovation visant à renforcer les capacités analytiques de l'équipe d'audit, en automatisant partiellement l'identification des transactions suspectes et en produisant des explications compréhensibles par les analystes métier.

1.4.2 Missions confiées

Les missions confiées dans le cadre de ce stage couvrent l'intégralité du cycle CRISP-DM :

- Compréhension du contexte métier de la détection de fraude financière en environnement d'audit ;

- Analyse exploratoire du corpus de transactions réelles (~200 000 opérations, ratio fraude 1 : 774);
- Conception et implémentation d'un pipeline de préparation des données (nettoyage, ingénierie de variables, gestion du déséquilibre via SMOTE);
- Développement et comparaison de trois approches de modélisation : régression logistique, forêt aléatoire et autoencodeur non supervisé;
- Mise en place d'une couche d'explicabilité double : méthodes SHAP et LIME, et génération d'explications en langage naturel via **Ollama**;
- Déploiement d'un pipeline reproductible via le script d'orchestration `run_all.py`, avec couverture de tests unitaires.

1.4.3 Présentation du projet

Le projet vise à construire une **chaîne de traitement complète et reproductible** pour la détection de transactions frauduleuses. Conduit selon la méthodologie **CRISP-DM**, il répond à la problématique suivante : *comment détecter efficacement les fraudes dans un corpus fortement déséquilibré, tout en produisant des explications compréhensibles pour les auditeurs ?*

La solution développée repose sur trois contributions complémentaires :

- **Modélisation multi-approches** : comparaison d'un modèle linéaire, d'un modèle ensembliste supervisé et d'un autoencodeur profond non supervisé, évalués sur des métriques adaptées au déséquilibre (F1-score, PR-AUC, rappel);
- **Gestion du déséquilibre** : stratégies de pondération des classes et de rééchantillonnage SMOTE pour traiter le ratio fraude/légitime de 1 : 774;
- **Explicabilité des décisions** : identification des variables les plus contributives via SHAP et LIME, et génération d'explications en langage naturel à destination des analystes métier via le LLM Ollama.

Conclusion

Ce premier chapitre a établi le cadre institutionnel et organisationnel dans lequel s'inscrit ce projet de fin d'études. À travers la présentation de PwC en tant que réseau international de référence, puis de PwC Tunisie et de son département Risk Assurance Services, nous avons mis en lumière l'environnement professionnel au sein duquel ce stage a été effectué, ainsi que le rôle stratégique de la cellule Audit IT. La problématique identifiée — détecter automatiquement les transactions frauduleuses dans un corpus réel fortement déséquilibré, tout en garantissant l'interprétabilité des résultats — et les missions confiées ont posé les bases nécessaires à la compréhension des enjeux du projet.

Le chapitre suivant sera consacré à la compréhension métier, première phase de la méthodologie CRISP-DM, dans laquelle nous formaliserons la problématique, les objectifs et les critères de succès du projet.

2 Compréhension métier

Introduction

Avant d'écrire la moindre ligne de code, il faut comprendre le problème que l'on cherche à résoudre — et surtout, le métier derrière les données. C'est l'essence de la première phase CRISP-DM. Ce chapitre pose donc le cadre : contexte de la fraude financière telle qu'on la rencontre en mission d'audit, formalisation de la problématique, objectifs opérationnels et critères de succès mesurables qui guideront toutes les décisions techniques ultérieures. Cette étape est non négociable dans un contexte d'audit IT : une définition floue des objectifs aurait conduit à des choix algorithmiques déconnectés des besoins réels des auditeurs.

2.1 Contexte de la détection de fraude financière

2.1.1 Enjeux économiques et opérationnels

La fraude financière représente une menace persistante et coûteuse pour les institutions. Si les chiffres globaux donnent le vertige — le rapport Nilson (2022) évalue à plusieurs dizaines de milliards de dollars les pertes annuelles liées à la fraude par carte bancaire —, c'est à l'échelle d'une mission d'audit que l'impact se fait le plus ressentir : transactions non reconnues, coûts de remboursement, investigations manuelles chronophages et responsabilité engagée des auditeurs. À cela s'ajoutent des contraintes réglementaires croissantes, comme la directive européenne PSD2 qui impose des mécanismes de surveillance renforcés des transactions.

Pour une équipe comme celle du Risk Assurance Services de PwC Tunisie, l'enjeu est double : détecter les anomalies avant qu'elles ne se propagent, et le faire de manière traçable et défendable dans un rapport d'audit.

2.1.2 Limites des systèmes basés sur des règles

Historiquement, la détection de fraude repose sur des règles métier définies manuellement par des experts : seuils de montant, listes noires de comptes suspects, patterns connus de détournement. Ces systèmes ont le mérite d'être explicables et rapides à déployer. Mais lors de l'analyse du corpus de ce stage, le constat a été frappant : le système naïf `isFlaggedFraud` présent dans les données n'identifie qu'un seul cas de fraude sur 258, soit un rappel de **0,39 %**. Trois limites fondamentales expliquent cet échec :

- **Rigidité** : les règles codifient les comportements connus, mais les fraudeurs adaptent continuellement leurs méthodes pour les contourner.
- **Maintenance coûteuse** : chaque nouvelle forme de fraude nécessite une intervention manuelle d'un expert pour mettre à jour les règles.
- **Faux positifs élevés** : les seuils statiques génèrent de nombreuses alertes injustifiées, surchargeant les équipes d'investigation et dégradant la confiance dans le système.

L'apprentissage automatique offre une alternative complémentaire : plutôt que d'encoder les patterns de fraude à la main, il les extrait automatiquement des données historiques, y compris

des interactions non linéaires entre variables que l'œil humain ne perçoit pas.

2.2 Présentation et formalisation du projet

2.2.1 Contexte du projet de fin d'études

Ce projet a été réalisé dans le cadre d'un stage de fin d'études axé sur la mise en œuvre d'un système de détection d'anomalies financières exploitant des techniques avancées d'apprentissage automatique. Le travail couvre l'intégralité du cycle de vie d'un projet data science, de la définition du problème au déploiement d'un pipeline reproductible.

2.2.2 Problématique

Problématique : Comment concevoir un pipeline reproductible, fondé sur la méthodologie CRISP-DM, capable de détecter efficacement les transactions financières frauduleuses dans un contexte de fort déséquilibre des classes, tout en produisant des explications compréhensibles pour les analystes métier ?

Cette problématique est structurée autour de deux défis techniques majeurs : (1) la **gestion du déséquilibre extrême** des classes — moins de 0,13 % des transactions sont frauduleuses, un ratio si faible qu'un modèle naïf prédisant systématiquement « légitime » afficherait 99,87 % de précision tout en ne détectant aucune fraude — et (2) l'**interprétabilité** des décisions, rendue possible grâce aux méthodes SHAP, LIME et à la génération d'explications textuelles par le modèle de langage Ollama.

2.2.3 Objectifs du projet

Les objectifs opérationnels définis en début de projet sont les suivants :

1. Construire une **chaîne de traitement complète et reproductible** couvrant toutes les phases CRISP-DM.
2. Réaliser une **analyse exploratoire approfondie** des données transactionnelles.
3. Concevoir et appliquer un **pipeline de préparation des données** robuste, préservant l'intégrité temporelle et évitant toute fuite d'information (*data leakage*).
4. **Entraîner et comparer** au moins trois approches de détection : baseline linéaire, modèle ensembliste supervisé et autoencoder non supervisé.
5. Évaluer les modèles avec des **métriques adaptées au déséquilibre** : rappel, précision, F1-score, courbe PR et PR-AUC.
6. Produire des **explications interprétables** des décisions (SHAP, LIME, Ollama).
7. Livrer un **code modulaire, documenté et testé** facilitant la maintenance et l'extension.

2.2.4 Critères de succès

TABLE 2.1 – Critères de succès du projet

Critère	Valeur cible	Justification
Recall (test)	> 0,70	Priorité à la détection des fraudes
F1-score (test)	> 0,70	Équilibre précision/rappel
PR-AUC (test)	> 0,75	Synthèse du tradeoff
Reproductibilité	100 %	Pipeline exécutable via <code>run_all.py</code>
Couverture tests	> 80 %	Robustesse du code

2.3 Cadre méthodologique : CRISP-DM

2.3.1 Présentation de la méthodologie

CRISP-DM (*Cross-Industry Standard Process for Data Mining*) est un cadre méthodologique ouvert proposé en 1996 par un consortium réunissant SPSS, Daimler-Benz, NCR et OHRA. Son objectif : structurer de façon reproductible les projets de fouille de données, quelle que soit l'industrie ou les outils utilisés. Il organise le travail en **six phases inter-dépendantes** formant un cycle itératif, du cadrage métier jusqu'à la mise en production.

Les six phases de CRISP-DM sont :

1. **Business Understanding** — Compréhension métier et définition des objectifs.
2. **Data Understanding** — Collecte, exploration et évaluation de la qualité des données.
3. **Data Preparation** — Sélection, nettoyage, construction et formatage des données.
4. **Modeling** — Sélection, construction et ajustement des modèles.
5. **Evaluation** — Évaluation des résultats au regard des objectifs métier.
6. **Deployment** — Mise en production et documentation.

2.3.2 Justification du choix de CRISP-DM

Plusieurs méthodologies structurées existent pour conduire des projets de data science. Le tableau ci-dessous les compare pour justifier le choix retenu.

TABLE 2.2 – Comparaison des principales méthodologies de data science

Méthodologie	Points forts	Limites	Adéquation
CRISP-DM	Agnostique outils ; cycle itératif ; couverture complète (métier → déploiement)	Moins adapté aux projets agiles purs	Excellent
SEMMA (SAS)	Simple et concis	Lié à SAS ; absence de phase déploiement	Insuffisant
KDD	Fondement académique solide	Peu flexible ; orienté recherche	Partiel
Agile DS	Itérations courtes, DevOps	Suppose équipe multidisciplinaire et CI/CD	Inadapté

Le choix de CRISP-DM pour ce projet n’était pas arbitraire. Plusieurs raisons concrètes l’ont motivé :

- **Orientation métier explicite** : avant de toucher aux données, CRISP-DM impose de formaliser les objectifs et les critères de succès — étape non négociable en contexte d’audit IT.
- **Couverture complète du cycle** : contrairement à SEMMA ou KDD, CRISP-DM couvre aussi bien la compréhension des données que le déploiement du pipeline.
- **Caractère itératif** : la préparation des données et la modélisation ne sont pas séquentielles en pratique. Les allers-retours entre phases sont formalisés et légitimés par CRISP-DM.
- **Indépendance des outils** : CRISP-DM étant agnostique, il s’adapte sans difficulté à une stack Python / scikit-learn / PyTorch / Ollama.
- **Lisibilité pour les équipes PwC** : CRISP-DM étant largement adopté dans les cabinets de conseil, les encadrants entreprise ont pu suivre la progression du projet sans avoir à maîtriser les détails techniques.

2.3.3 Application au projet

L’ensemble du rapport est structuré selon ces six phases. Le caractère itératif de CRISP-DM se manifeste notamment dans les allers-retours entre l’analyse exploratoire (phase 2) et la préparation des données (phase 3), ainsi qu’entre la modélisation (phase 4) et l’évaluation (phase 5) lors du réglage des seuils de détection.

Conclusion

Ce chapitre a établi le contexte métier du projet, la problématique à résoudre et les objectifs à atteindre selon le cadre CRISP-DM. Deux défis principaux guideront toutes les décisions techniques : la gestion du déséquilibre extrême des classes (ratio 1 :774) et la nécessité de produire des décisions interprétables pour les auditeurs. Le choix de CRISP-DM comme méthodologie directrice a été justifié par sa couverture complète du cycle de vie, sa nature itérative et son indépendance vis-à-vis des outils. Le chapitre suivant présente les données sur lesquelles repose l'intégralité du travail.

3 Compréhension des données

Introduction

La phase de compréhension des données constitue le socle de tout projet de data science. Elle précède et conditionne chaque décision de modélisation : un modèle ne peut être meilleur que les données sur lesquelles il est entraîné. Avant d'appliquer le moindre algorithme, il est indispensable de saisir la structure, les limites et les opportunités du corpus disponible. Ce chapitre correspond au notebook `01_data_understanding.ipynb` du projet et présente le jeu de données, son analyse exploratoire, l'étude de l'existant, ainsi que l'état de l'art des approches de détection d'anomalies financières.

3.1 Présentation du jeu de données

3.1.1 Origine et description

Le jeu de données utilisé dans ce projet provient d'une **mission d'audit IT réalisée au sein de PwC Tunisie**. Il regroupe des transactions financières issues du système d'information d'un client. Pour des raisons de confidentialité, toutes les informations permettant d'identifier les parties concernées ont été anonymisées.

3.1.2 Caractéristiques générales

Le corpus est constitué d'environ **200 000 transactions financières** enregistrées sur une période couvrant 743 heures (soit environ 31 jours). Parmi ces transactions, seules **258 sont frauduleuses**, soit un taux de fraude de **0,129 %**. Ce déséquilibre extrême (ratio 1 : 774) constitue le défi central du projet.

3.1.3 Description des variables

TABLE 3.1 – Description des variables du jeu de données

Variable	Type	Description
step	Entier	Pas de temps (1 = 1 heure, max 743)
type	Catégorie	Type d'opération (5 modalités)
amount	Réel	Montant de la transaction
nameOrig	Chaîne	Identifiant du compte émetteur
oldbalanceOrg	Réel	Solde émetteur avant transaction
newbalanceOrig	Réel	Solde émetteur après transaction
nameDest	Chaîne	Identifiant du compte destinataire
oldbalanceDest	Réel	Solde destinataire avant transaction
newbalanceDest	Réel	Solde destinataire après transaction
isFraud	Binaire	Variable cible : 1 = fraude
isFlaggedFraud	Binaire	Indicateur système naïf existant

3.1.4 Types de transactions

Cinq types de transactions sont présents dans le jeu de données :

TABLE 3.2 – Distribution des types de transactions et taux de fraude

Type	Fréquence	Taux de fraude
CASH_IN	~35 %	0,000 %
CASH_OUT	~35 %	0,181 %
DEBIT	~1 %	0,000 %
PAYMENT	~22 %	0,000 %
TRANSFER	~7 %	0,775 %

Observation clé : *seuls les types TRANSFER et CASH_OUT présentent des cas de fraude.* Cette information sera directement exploitée lors de l'ingénierie de variables (création de la feature `is_transfer_or_cashout`).

3.2 Analyse exploratoire des données (EDA)

3.2.1 Déséquilibre des classes

Le déséquilibre est extrême : les transactions frauduleuses représentent **0,129 %** du corpus. Un modèle prédisant systématiquement « légitime » atteindrait une exactitude de 99,87 % tout en ne détectant **aucune fraude**. C'est pourquoi l'exactitude (*accuracy*) est une métrique totalement inadaptée à ce problème, et que les métriques de rappel, précision et F1-score ont été retenues comme indicateurs de succès.

3.2.2 Distribution des montants

La variable `amount` présente une asymétrie (*skewness*) très élevée (**30,80**), indiquant une distribution fortement concentrée sur de petites valeurs avec de rares transactions à montant très élevé.

TABLE 3.3 – Statistiques descriptives de la variable `amount`

Statistique	Valeur
Minimum	0,00
Médiane	~ 74 000
Moyenne	~ 179 862
Maximum	~ 92 445 517
Skewness	30,80

Cette propriété justifie l'application d'une transformation logarithmique $\log(1 + x)$ lors de la préparation des données pour corriger la skewness et améliorer les performances des modèles linéaires.

3.2.3 Analyse temporelle

L'analyse de la variable `step` révèle une **saisonnalité des fraudes**. Les heures à risque élevé identifiées dans l'EDA sont : $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 23\}$ — soit les heures nocturnes et de début de matinée. Le ratio de fraude durant ces heures est de **10,63 fois supérieur** à celui des heures ordinaires. Cette observation conduit à la création d'une variable indicatrice `high_risk_hour`.

3.2.4 Analyse des soldes et détection de leakage

L'analyse des variables de soldes révèle des incohérences pour les transactions frauduleuses : `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest` et `newbalanceDest` présentent des soldes nuls anormaux. Ces variables sont des **indicateurs trop directs de la fraude** et pourraient causer une fuite d'information (*data leakage*) si elles étaient directement incluses dans les modèles. À la place, une variable dérivée `balance_diff_orig` est créée, avec une corrélation de **0,3662** avec la variable cible `isFraud`.

3.2.5 Baseline du système existant

Le système existant (`isFlaggedFraud`) sert de référence basse. Ses performances illustrent l'inadéquation des approches par règles simples :

TABLE 3.4 – Performances de la baseline `isFlaggedFraud`

Métrique	Valeur
Recall	0,0039
Precision	1,0000
F1-Score	0,0077

Ce système ne détecte qu'une infime fraction des fraudes réelles — une seule transaction sur 258 —, confirmant la nécessité d'une approche par apprentissage automatique.

3.3 Étude de l'existant et état de l'art

3.3.1 Approches supervisées

Les approches supervisées nécessitent des étiquettes (*labels*) pour chaque transaction. Parmi les plus utilisées pour la détection de fraude :

- **Régression logistique** : baseline linéaire, faible coût computationnel, bonne interprétabilité.
- **Forêt aléatoire** : ensembliste, robuste au bruit, gère bien les interactions non linéaires entre variables.
- **Gradient Boosting (XGBoost, LightGBM)** : performances état de l'art sur données tabulaires.
- **Réseaux de neurones feedforward** : puissants mais nécessitent beaucoup de données labellisées.

3.3.2 Approches non supervisées

Lorsque les étiquettes sont rares ou indisponibles, les approches non supervisées permettent de détecter des patterns inhabituels sans connaissance préalable des fraudes :

- **Isolation Forest** : isole les anomalies par partitions aléatoires successives.
- **One-Class SVM** : délimite la frontière de la normalité dans l'espace des features.
- **Autoencoder** : apprend à reconstruire les données normales ; une erreur de reconstruction élevée signale une anomalie.
- **Autoencoders variationnels (VAE)** : modélisation probabiliste de la distribution normale, plus robuste pour les données continues.

3.3.3 Approches retenues pour ce projet

Pour ce projet, trois approches complémentaires ont été retenues, couvrant le spectre supervisé / non supervisé :

1. **Régression logistique** — baseline supervisée simple, pour établir une référence linéaire solide.
2. **Forêt aléatoire** — référence supervisée robuste, capable de capturer les interactions non linéaires entre variables.
3. **Autoencoder** — approche non supervisée orientée reconstruction, utile lorsque les labels sont limités ou lorsqu'on souhaite modéliser les patterns normaux sans dépendre des exemples de fraude.

Ce choix vise à couvrir plusieurs angles d'attaque du problème, afin de disposer d'une comparaison objective et d'une complémentarité méthodologique.

Conclusion

L'analyse exploratoire a mis en évidence les caractéristiques clés du jeu de données : déséquilibre extrême des classes (1 : 774), asymétrie des montants ($\text{skewness} = 30,80$), saisonnalité temporelle des fraudes (ratio 10,63× la nuit) et risque de fuite d'information sur les variables de soldes. Ces observations ne sont pas de simples curiosités statistiques — elles orientent directement chaque décision de préparation et de modélisation décrite dans les chapitres suivants. L'inadéquation flagrante du système de règles existant ($F1 = 0,0077$) confirme l'utilité d'une approche par machine learning. Le chapitre suivant décrit en détail les opérations de préparation des données.

4 Préparation des données

Introduction

Préparer les données, c'est poser les fondations sur lesquelles reposent tous les modèles à venir. J'ai rapidement réalisé que les décisions prises ici auraient des répercussions directes sur la validité de l'évaluation finale. L'enjeu central n'était pas seulement technique, mais méthodologique : comment extraire des signaux pertinents des variables brutes, tout en évitant d'introduire sans le vouloir de l'information du futur dans l'apprentissage — ce que l'on appelle le *data leakage* ? Ce chapitre détaille chaque choix effectué et les justifications empiriques qui les sous-tendent, telles qu'elles ressortent des notebooks `01_data_understanding.ipynb` et `02_data_preparation.ipynb`.

4.1 Pipeline de préparation

4.1.1 Vue d'ensemble

Le pipeline de préparation suit les étapes définies dans le module `src/preprocessing/preprocess`. Ces étapes sont appliquées dans un ordre strict pour garantir l'étanchéité des partitions et l'absence de fuite d'information :

1. Suppression des variables à risque de leakage
2. Ingénierie de variables (*feature engineering*)
3. Division stratifiée du corpus (train / validation / test)
4. Normalisation (StandardScaler, *fit* sur train uniquement)
5. Gestion du déséquilibre (pondération et SMOTE)

4.1.2 Suppression des variables à risque de leakage

La première décision consiste à identifier et éliminer les variables qui introduiraient de l'information future ou qui révéleraient directement la classe cible. Ces suppressions sont motivées empiriquement par l'analyse exploratoire :

TABLE 4.1 – Variables supprimées et justifications

Variable	Justification
nameOrig, nameDest	Identifiants nominaux non encodables sans risque de sur-apprentissage
oldbalanceOrig, newbalanceOrig	Corrélation directe avec la fraude ; fuite d'information avérée
oldbalanceDest, newbalanceDest	Idem ; révèlent les patterns frauduleux de manière trop évidente
isFlaggedFraud	Label proxy du système existant ; ne doit pas alimenter l'apprentissage

4.2 Ingénierie de variables

4.2.1 Variables temporelles

La variable `step` (pas de temps en heures) est décomposée en trois variables cycliques qui capturent des patterns à différentes granularités temporelles :

- `hour` : heure dans la journée (0–23), calculée par `step mod 24` ;
- `day` : jour de simulation (0–30), calculé par `⌊step/24⌋ mod 31` ;
- `week` : semaine de simulation (0–4), calculée par `⌊step/168⌋`.

4.2.2 Variables comportementales et de risque

Cinq variables dérivées sont construites à partir des observations de l'EDA :

TABLE 4.2 – Variables dérivées construites lors du feature engineering

Variable	Formule / Logique	Signal mesuré
<code>high_risk_hour</code>	1 si <code>hour</code> $\in \{0, \dots, 9, 23\}$	Ratio fraude 10,63× supérieur
<code>is_transfer_or_cash</code>	<code>type</code> $\in \{\text{TRANSFER, CASH_OUT}\}$	Seuls types avec fraudes réelles
<code>balance_diff_orig</code>	$\frac{\text{newbalanceOrig} - \text{oldbalanceOrig}}{\text{oldbalanceOrig}}$	Corrélation 0,3662 avec <code>isFraud</code>
<code>dest_zero_balance</code>	1 si <code>oldbalanceDest</code> = 0	Taux fraude 2,52 % vs 0,13 % global
<code>log_amount</code>	$\log(1 + \text{amount})$	Corrige skewness = 30,80

Chacune de ces variables est justifiée empiriquement par l'EDA — aucun choix n'est arbitraire.

4.2.3 Encodage one-hot

La variable catégorielle `type` (5 modalités) est encodée par **one-hot encoding** (`drop_first=False`), produisant les colonnes `type_CASH_IN`, `type_CASH_OUT`, `type_DEBIT`, `type_PAYMENT` et `type_TRANSFER`. L'absence de suppression d'une colonne (`drop_first=False`) est volontaire : avec un arbre de décision ou un autoencoder, toutes les modalités sont nécessaires.

4.2.4 Ensemble final des variables

Après feature engineering, le jeu de données comprend **14 variables** :

TABLE 4.3 – Ensemble final des 14 variables du modèle

Catégorie	Variables	Traitement
Temporelles	step, hour, day, week	StandardScaler
Montant	log_amount	StandardScaler
Comportement	balance_diff_orig	StandardScaler
Binaires	high_risk_hour, is_transfer_or_cashout, dest_zero_balance	—
Type (OHE)	type_CASH_IN, type_CASH_OUT, type_DEBIT, type_PAYMENT, type_TRANSFER	—

4.3 Division et normalisation

4.3.1 Division stratifiée

Le corpus est divisé en trois sous-ensembles de manière **stratifiée** sur la variable cible, afin de conserver la proportion de fraudes dans chaque partition — essentiel lorsque les classes sont aussi déséquilibrées.

TABLE 4.4 – Division stratifiée du corpus

Ensemble	Taille	Fraudes	Rôle
Entraînement	139 999	181	Apprentissage des modèles
Validation	30 000	38	Réglage des hyperparamètres et seuils
Test	30 001	39	Évaluation finale (jamais consulté avant)

Principe d’étanchéité : L’ensemble de test n’est jamais consulté avant l’évaluation finale, conformément aux bonnes pratiques anti-*data snooping*. Ce principe a été respecté tout au long du projet.

4.3.2 Normalisation

Le **StandardScaler** est ajusté (*fit*) exclusivement sur l’ensemble d’entraînement, puis appliqué (*transform*) sur les trois partitions. Cette discipline stricte garantit l’absence de leakage statistique : aucune information sur la distribution du test ne s’infiltrer dans les transformations appliquées à l’entraînement.

4.4 Gestion du déséquilibre

4.4.1 Pondération des classes

Pour les modèles supervisés, des poids de classes sont calculés automatiquement (formule *sklearn balanced*) :

$$w_c = \frac{n_{\text{total}}}{n_{\text{classes}} \times n_c}$$

Avec 139 999 transactions d’entraînement dont 181 fraudes, les poids obtenus sont approximativement $w_0 \approx 0,50$ et $w_1 \approx 386,7$. Cela revient à donner à chaque transaction frauduleuse un poids environ 386 fois supérieur à une transaction légitime lors du calcul de la fonction de perte — compensant ainsi le déséquilibre du corpus.

4.4.2 SMOTE

Pour les variantes avec rééchantillonnage, **SMOTE** (*Synthetic Minority Over-sampling Technique*) génère des exemples synthétiques de la classe minoritaire par interpolation dans l’espace des features. Le principe : pour chaque transaction frauduleuse, SMOTE choisit aléatoirement k voisins les plus proches dans l’espace des features, puis crée de nouveaux exemples sur les segments reliant la transaction à ses voisins.

Il est appliqué **uniquement sur l’ensemble d’entraînement** (jamais sur validation ou test), afin de ne pas contaminer l’évaluation avec des exemples synthétiques.

4.4.3 Ensemble pour l’autoencoder

Pour l’autoencoder non supervisé, un sous-ensemble `X_train_normal` est extrait : il contient **uniquement les 139 818 transactions légitimes** de l’entraînement (0 fraude). Cela permet au modèle d’apprendre exclusivement la distribution normale — principe fondateur de la détection d’anomalies par reconstruction.

Conclusion

La préparation des données produit un jeu de **14 variables représentatives** des patterns de fraude, en respectant strictement l’étanchéité entre les partitions. Les 7 variables dérivées ont chacune été validées par une mesure empirique issue de l’EDA — aucune n’est arbitraire. La discipline anti-leakage (StandardScaler fit sur train seulement, SMOTE appliqué uniquement sur le train, test jamais consulté) garantit que l’évaluation finale reflète les véritables performances sur des données inédites. Ce socle solide permet d’entraîner des modèles fiables et comparables, décrits dans le chapitre suivant.

5 Modélisation

Introduction

Une fois les données préparées, vient la phase de construction des modèles. Plutôt que de choisir d'emblée l'approche la plus sophistiquée, j'ai opté pour une progression logique : commencer par un modèle simple servant de référence, complexifier si les performances le justifient, puis compléter avec une perspective non supervisée. Cette stratégie permet d'évaluer la valeur ajoutée réelle de chaque niveau de complexité et d'éviter un sur-ingénierie inutile. Ce chapitre décrit les trois architectures développées et les décisions qui ont guidé leur conception, à travers les notebooks `03_baseline_models.ipynb` et `04_autoencoder.ipynb`.

5.1 Régression logistique

5.1.1 Principe et rôle dans le projet

La régression logistique est la **baseline** — le modèle de référence que tout algorithme plus complexe doit surpasser pour justifier sa complexité. Elle produit une probabilité d'appartenance à la classe fraude via la fonction sigmoïde :

$$P(\text{fraude} \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

Sa faible capacité expressive est ici un avantage : si la régression logistique détecte déjà une grande partie des fraudes, cela signifie que le signal est linéairement séparable, ce qui simplifie l'ensemble de la chaîne. Dans le cas contraire, cela justifie le passage à des modèles non linéaires.

5.1.2 Hyperparamètres et configurations évaluées

TABLE 5.1 – Hyperparamètres de la régression logistique

Paramètre	Valeur	Justification
C	0,1	Régularisation forte (peu de fraudes)
solver	lbfgs	Adapté aux datasets de taille moyenne
class_weight	balanced	Correction automatique du déséquilibre
max_iter	1000	Assure la convergence

Deux configurations sont comparées :

- **LR_balanced** : entraînée sur `X_train` avec `class_weight='balanced'` ;
- **LR_smote** : entraînée sur `X_smote` (données équilibrées par SMOTE) sans pondération explicite.

5.1.3 Réglage du seuil de décision

À seuil par défaut (0,5), LR_balanced maximise fortement le rappel mais au prix d'une précision très faible. Une recherche de seuil sur l'ensemble de validation est donc réalisée à partir des probabilités produites par le modèle. Les seuils optimaux retenus sont **0,9986** pour LR_balanced et **0,9621** pour LR_smote. Après ce recalibrage, LR_smote devient la meilleure variante logistique avec un rappel de **0,6923**, une précision de **0,7297** et un F1-score de **0,7105** sur l'ensemble de test.

5.2 Forêt aléatoire

5.2.1 Principe et avantages

La forêt aléatoire constitue la **référence supervisée robuste** de ce projet. Son principe repose sur la diversité : entraîner un grand nombre d'arbres de décision sur des sous-échantillons distincts des données (*bootstrap*), puis agréger leurs prédictions. Cette diversité rend l'ensemble résistant au sur-apprentissage (*overfitting*) et capable de capturer des interactions non linéaires entre variables que la régression logistique ne peut pas modéliser.

La prédiction finale est la **moyenne des probabilités** émises par chaque arbre individuel, ce qui produit des scores bien calibrés.

5.2.2 Hyperparamètres

TABLE 5.2 – Hyperparamètres de la forêt aléatoire

Paramètre	Valeur	Justification
n_estimators	300	Stabilité des estimations
max_depth	10	Limite l'overfitting sur peu de fraudes
min_samples_leaf	5	Évite les feuilles trop petites
class_weight	balanced	Correction du déséquilibre

Deux variantes sont comparées :

- **RF_balanced** : entraînée sur X_train avec class_weight='balanced' ;
- **RF_smote** : entraînée sur X_smote, sans pondération explicite.

5.2.3 Réglage du seuil et importance des variables

Les seuils optimaux retenus après recherche sur validation sont **0,6235** pour RF_balanced et **0,6291** pour RF_smote. La variante RF_smote obtient les meilleures performances supervisées globales sur le test : rappel de **0,7949**, précision de **0,8158** et F1-score de **0,8052**.

La forêt aléatoire permet également de calculer l'importance de chaque variable via la réduction moyenne d'impureté (*Mean Decrease in Impurity*). Sur RF_smote, les cinq variables les plus influentes sont :

TABLE 5.3 – Importance des variables (RF_smote)

Variable	Importance
balance_diff_orig	~0,385
hour	~0,118
dest_zero_balance	~0,115
type_TRANSFER	~0,089
log_amount	~0,081

Cette hiérarchie confirme que le comportement du compte émetteur, la temporalité et la nature du virement sont les signaux les plus discriminants — en cohérence avec l’EDA.

5.3 Autoencoder

5.3.1 Principe de détection par reconstruction

L’autoencoder repose sur une idée intuitive : **apprendre à compresser puis à reconstruire fidèlement les données normales**. Si une transaction légitime peut être reproduite depuis un espace latent de petite dimension, c’est qu’elle s’inscrit dans des patterns réguliers. Une transaction frauduleuse — jamais vue à l’entraînement — sera en revanche mal reconstruite, trahissant son caractère inhabituel par une erreur de reconstruction élevée.

Principe de l’autoencoder pour la détection d’anomalies :

- **Entraînement** : uniquement sur les transactions légitimes → le modèle apprend à reconstruire fidèlement les patterns normaux.
- **Inférence** : sur une transaction frauduleuse (pattern jamais vu), l’erreur de reconstruction est élevée car le modèle ne sait pas comment la reproduire.
- **Décision** : si l’erreur de reconstruction (MSE) dépasse un seuil θ , la transaction est classée comme anomalie.

5.3.2 Architecture du réseau

L’architecture retenue est un autoencoder symétrique à trois couches dans l’encodeur et trois dans le décodeur, avec un espace latent (*bottleneck*) de dimension 4 :

TABLE 5.4 – Architecture de l’autoencoder ($14 \rightarrow 10 \rightarrow 7 \rightarrow [4] \rightarrow 7 \rightarrow 10 \rightarrow 14$)

Partie	Couche	Dimension	Activation
Entrée	Input	14	—
Encodeur	Dense 1	10	ReLU + BatchNorm + Dropout(0,2)
	Dense 2	7	ReLU + BatchNorm + Dropout(0,2)
	Bottleneck	4	ReLU
Décodeur	Dense 3	7	ReLU + BatchNorm + Dropout(0,2)
	Dense 4	10	ReLU + BatchNorm + Dropout(0,2)
Sortie	Dense 5	14	Linéaire

Les techniques de régularisation employées sont :

- **BatchNorm** (normalisation par lots) : stabilise l’apprentissage ;
- **Dropout** (taux 0,2) : réduit le sur-apprentissage ;
- **Régularisation L2** ($\lambda = 10^{-5}$) sur les poids denses.

5.3.3 Entraînement

TABLE 5.5 – Paramètres d’entraînement de l’autoencoder

Paramètre	Valeur
Données d’entraînement	139 818 transactions légitimes (0 fraude)
Validation interne	10 % du train normal
Fonction de perte	MSE (Mean Squared Error)
Optimiseur	Adam ($\alpha = 10^{-3}$)
Batch size	256
Epochs max	100 (EarlyStopping, patience = 10)
ReduceLROnPlateau	facteur 0,5, patience 5, min_lr 10^{-6}

L’entraînement s’arrête après **35 époques effectives** grâce à *EarlyStopping*. Le meilleur `val_loss` observé est de **0,145024** et le temps d’entraînement est de **45,2 secondes**. Le modèle comporte **664 paramètres** entraînaibles, ce qui reste très léger au regard de la taille du corpus.

5.3.4 Détermination du seuil de détection

Une première borne est estimée par le **95^e percentile** de l’erreur de reconstruction sur les transactions normales d’entraînement ($\theta_{p95} = 0,403750$). Ce seuil est utile comme repère initial, mais trop permissif pour une décision finale.

Le seuil optimal est ensuite trouvé sur l’ensemble de validation (`X_val`) en balayant 200 valeurs entre le 50^e et le 99,9^e percentile des erreurs de reconstruction. Pour chaque valeur, le F1-score est calculé. Le seuil maximisant ce score est retenu : $\theta = 1,687408$.

5.3.5 Visualisation de l'espace latent

La projection PCA du bottleneck (dimension 4) en 2D permet de vérifier qualitativement que les fraudes de validation tendent à se positionner **en périphérie des zones de forte densité normale** — confirmant la pertinence du mécanisme de reconstruction comme score d'anomalie.

5.4 Récapitulatif des configurations

TABLE 5.6 – Récapitulatif des configurations de modèles construites

Configuration	Type	Données train	Gestion déséqui- libre
LR_balanced	Supervisé li- néaire	X_train	class_weight
LR_smote	Supervisé li- néaire	X_smote	SMOTE
RF_balanced	Supervisé ensem- bliste	X_train	class_weight
RF_smote	Supervisé ensem- bliste	X_smote	SMOTE
Autoencoder	Non supervisé	X_train_normale	Données normales seules

Conclusion

Ce chapitre a décrit la conception et l'implémentation des cinq configurations de modèles étudiées. Les variantes supervisées montrent que le choix du seuil de décision influence fortement le compromis précision/rappel, et que RF_smote s'impose comme la meilleure configuration supervisée. L'autoencoder complète ce dispositif par une logique non supervisée fondée sur la reconstruction, particulièrement utile lorsque les labels sont rares. Les premiers résultats sur l'ensemble de validation suggèrent que la forêt aléatoire dépasse les objectifs fixés (recall > 0,70, F1 > 0,70). L'évaluation comparative détaillée sur l'ensemble de test, ainsi que les mécanismes d'explicabilité, sont présentés dans le chapitre suivant.

6 Évaluation et interprétabilité

Introduction

Avoir un modèle qui fonctionne sur les données d'entraînement ne suffit pas. L'évaluation sert à répondre à deux questions distinctes : le modèle détecte-t-il réellement les fraudes (évaluation quantitative sur métriques adaptées au déséquilibre) ? Et ses décisions sont-elles compréhensibles et défendables pour un auditeur (évaluation qualitative via SHAP, LIME et Ollama) ? Ces deux dimensions sont indissociables dans un contexte professionnel où les alertes doivent non seulement être justes, mais aussi justifiées. Ce chapitre synthétise les résultats issus des notebooks `03_baseline_models.ipynb`, `04_autoencoder.ipynb`, `05_ollama_integration` et `06_shap_lime.ipynb`.

6.1 Métriques d'évaluation

6.1.1 Choix et justification des métriques

Dans un contexte de fort déséquilibre, l'exactitude (*accuracy*) n'est pas pertinente : un modèle qui prédit systématiquement « légitime » obtiendrait 99,87 % d'exactitude tout en ne détectant aucune fraude. Les métriques retenues sont :

- **Rappel (Recall)** : fraction des fraudes réelles correctement détectées. *Priorité 1* car manquer une fraude a un coût financier et réputationnel direct.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Précision (Precision)** : fraction des alertes qui sont réellement des fraudes. *Priorité 2* car trop de faux positifs surchargent les équipes d'investigation.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **F1-Score** : moyenne harmonique de la précision et du rappel. Synthèse unique du compromis.

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **PR-AUC** (*Area Under the Precision-Recall Curve*) : synthétise le tradeoff précision/rappel sur l'ensemble des seuils possibles — robuste au déséquilibre des classes.

6.1.2 Résultats comparatifs sur l'ensemble de test

TABLE 6.1 – Comparaison des performances sur l'ensemble de test

Modèle	Seuil	Recall	Precision	F1-Score	PR-AUC
Baseline (isFlaggedFraud)	N/A	0,0039	1,0000	0,0077	N/A
LR_balanced	0,9986	0,6410	0,6579	0,6494	0,7044
LR_smote	0,9621	0,6923	0,7297	0,7105	0,7393
RF_balanced	0,6235	0,7692	0,8333	0,8000	0,7794
RF_smote	0,6291	0,7949	0,8158	0,8052	0,8405
Autoencoder	1,6874	0,3590	0,6087	0,4516	0,3734

Note : les seuils sont sélectionnés sur la validation, puis figés pour l'évaluation finale sur l'ensemble de test — jamais consulté lors de la phase d'optimisation.

6.1.3 Analyse de la matrice de confusion

La matrice de confusion permet d'analyser précisément les types d'erreurs :

- **Faux négatifs (FN)** : fraudes manquées — coût financier direct, impact réputationnel, responsabilité de l'auditeur.
- **Faux positifs (FP)** : alertes injustifiées — surcharge des équipes d'investigation, coût opérationnel.

La forêt aléatoire avec SMOTE offre le meilleur équilibre sur l'ensemble de test : **31 fraudes détectées sur 39** pour seulement **7 faux positifs**. La variante RF_balanced reste très proche (30 TP, 6 FP), tandis que LR_smote conserve un compromis acceptable (27 TP, 10 FP). L'autoencoder, en revanche, ne détecte que **14 fraudes sur 39** avec **9 faux positifs** — moins performant que les approches supervisées, mais conservant un intérêt lorsque les labels sont rares ou indisponibles.

6.1.4 Discussion et sélection du modèle

Tous les critères de succès définis en phase 1 sont atteints pour la forêt aléatoire avec SMOTE :

TABLE 6.2 – Vérification des critères de succès pour RF_smote

Critère	Cible	Résultat	Statut
Recall	> 0,70	0,7949	Atteint
F1-Score	> 0,70	0,8052	Atteint
PR-AUC	> 0,75	0,8405	Atteint

6.2 Interprétabilité des modèles

6.2.1 Méthodes SHAP

L'un des reproches fréquents adressés aux modèles d'apprentissage automatique est leur opacité. Dans un contexte d'audit, un analyste ne peut se contenter d'une alerte : il lui faut une justification claire des variables qui ont conduit le modèle à signaler une transaction.

SHAP (*SHapley Additive exPlanations*) est une méthode d'interprétabilité fondée sur la théorie des jeux coopératifs. Elle attribue à chaque variable une valeur représentant sa contribution marginale à la prédiction, calculée en moyennant l'impact de la variable sur toutes les combinaisons possibles de features. Cette approche garantit additivité, cohérence et absence de biais entre les variables.

Pour la forêt aléatoire, un explainer **TreeSHAP** est utilisé — plus rapide que l'approche agnostique et exact pour les modèles arborescents. Les résultats montrent que `balance_diff_orig`, `is_transfer_or_cashout` et `log_amount` sont les variables à plus forte contribution, en cohérence avec l'analyse exploratoire et l'importance des variables de la forêt.

Deux niveaux d'analyse SHAP sont produits :

- **Global** : beeswarm plot montrant la distribution des valeurs SHAP sur l'ensemble du test, permettant d'identifier les variables globalement les plus influentes.
- **Local** : waterfall plot pour une transaction spécifique, montrant comment chaque feature pousse la prédiction vers « fraude » ou « légitime ».

6.2.2 Méthodes LIME

LIME (*Local Interpretable Model-agnostic Explanations*) explique chaque prédiction individuellement en perturbant légèrement la transaction d'entrée et en observant comment la prédiction varie. Il approxime localement le comportement du modèle complexe par un modèle linéaire simple, interprétable par construction, valable uniquement dans le voisinage de l'exemple analysé.

LIME est particulièrement utile pour expliquer un cas suspect spécifique à un analyste, sans avoir besoin de comprendre le comportement global du modèle. Contrairement à SHAP, il est **agnostique** à l'architecture du modèle et peut s'appliquer à n'importe quel algorithme — y compris l'autoencoder, pour lequel SHAP n'est pas nativement adapté.

6.2.3 Comparaison SHAP vs LIME

TABLE 6.3 – Comparaison des méthodes d’explicabilité

Critère	SHAP	LIME
Scope	Global + Local	Local uniquement
Fondement	Théorie des jeux	Approximation locale linéaire
Fidélité	Élevée (TreeSHAP exact)	Variable (approximation)
Rapidité	Rapide (TreeSHAP)	Lent (nombreuses perturbations)
Agnosticisme	Méthodes spécifiques disponibles	Entièrement agnostique

6.3 Intégration Ollama

6.3.1 Pourquoi un LLM local ?

Ollama est un outil permettant d’exécuter des grands modèles de langage (LLM) **entièrement en local**, sans envoyer de données vers des services cloud externes. Ce point était non négociable dans le cadre de ce projet : les transactions financières auditées sont confidentielles et ne peuvent pas quitter l’environnement sécurisé du client. L’intégration avec Ollama permet de produire des explications en langage naturel — directement lisibles par un auditeur sans formation en machine learning — sans compromettre la confidentialité des données.

6.3.2 Architecture de l’intégration

Le module `src/llm_integration/llm_helper.py` encapsule la logique d’appel au modèle de langage. Pour chaque transaction signalée comme anormale, les éléments suivants sont transmis au LLM dans un prompt structuré :

- Le score d’anomalie ou d’erreur de reconstruction de l’autoencoder ;
- Les valeurs des variables les plus contributives (issues de SHAP/LIME) ;
- Le contexte métier de la transaction (type, montant, heure, différence de solde).

Le LLM génère une explication structurée en quatre points : type de transaction à risque, heure suspecte, comportement anormal des soldes, et recommandation d’action.

6.3.3 Difficultés rencontrées

- **Instabilité des seuils de l’autoencoder** : de légères variations du seuil peuvent significativement modifier le ratio précision/rappel, nécessitant une optimisation soigneuse sur la validation.
- **Variabilité des explications LLM** : les explications Ollama peuvent varier selon la version du modèle utilisé et la qualité du prompt ; un prompt engineering rigoureux a été nécessaire.
- **Temps de calcul SHAP** : le calcul des valeurs SHAP pour l’ensemble de test complet nécessite plusieurs minutes sur CPU, ce qui a motivé la sauvegarde des tenseurs SHAP dans `outputs/models/shap_values_*.npy`.

Conclusion

L'évaluation confirme que la forêt aléatoire avec SMOTE constitue la **meilleure approche supervisée** pour ce jeu de données, avec un recall de 0,7949, un F1-score de 0,8052 et un PR-AUC de 0,8405 — tous au-dessus des cibles fixées. L'autoencoder apporte une perspective complémentaire non supervisée, utile lorsque les étiquettes sont limitées. La **double couche d'explicabilité** (SHAP/LIME + Ollama) répond à l'objectif d'interprétabilité en rendant les prédictions compréhensibles tant pour les data scientists que pour les analystes métier non techniciens. Le chapitre suivant présente les aspects de déploiement et les garanties de reproductibilité du pipeline complet.

7 Déploiement et reproductibilité

Introduction

Un modèle performant qui ne peut pas être réexécuté par quelqu'un d'autre n'a pas grande valeur dans un contexte professionnel. Pour ce projet, le déploiement signifie rendre l'ensemble du travail **reproductible de bout en bout**, maintenable par un futur contributeur et extensible si les besoins évoluent. Ce chapitre décrit l'architecture choisie, l'organisation des sorties, le script d'orchestration, les garanties de reproductibilité, la suite de tests unitaires mise en place, et les perspectives d'amélioration identifiées pour une éventuelle industrialisation du système.

7.1 Architecture du projet

7.1.1 Structure modulaire

Le projet est organisé en modules fonctionnels clairement délimités, chacun ayant une responsabilité unique (*separation of concerns*) :

TABLE 7.1 – Structure des modules du projet

Module	Responsabilité
src/preprocessing/	Chargement, nettoyage, normalisation, split
src/feature_engineering/	Création des 7 variables dérivées
src/models/	Modèles ML (LR, RF) et autoencoder PyTorch
src/pipeline/	Orchestration entraînement et inférence
src/utils/	Métriques, évaluation, utilitaires communs
src/visualization/	Génération de figures et graphiques
src/ollama_integration/	Génération d'explications LLM via Ollama
src/explainability/	Explicabilité SHAP et LIME
tests/	Tests unitaires (pytest)

Cette organisation garantit que chaque module peut être modifié, testé et remplacé indépendamment — un avantage décisif pour la maintenance et l'évolution du système.

7.1.2 Organisation des sorties

Le pipeline produit des artefacts organisés dans le répertoire `outputs/` :

- `outputs/models/` : artefacts sérialisés — poids de l'autoencoder, scalers, seuils optimaux, poids des classes, scores AE (`ae_scores_val/test.npy`) et tenseurs SHAP sauvegardés (`shap_values_*.npy`);
- `outputs/reports/` : rapports JSON des phases exécutées (`eda_report.json`, `prep_report.json`, `baseline_report.json`, `autoencoder_report.json`);

- `outputs/figures/` : courbes de performance, matrices de confusion, distributions, beeswarm SHAP, projection latente de l'AE ;
- `outputs/anomalies_report.csv` : liste des transactions signalées avec leurs scores et niveaux de risque ;
- `outputs/explanations.json` : explications textuelles des anomalies générées par Ollama.

7.2 Orchestration et reproductibilité

7.2.1 Script d'orchestration

Le script `run_all.py` constitue le point d'entrée unique du pipeline. Il valide d'abord les imports des modules `src/`, puis exécute séquentiellement les six notebooks (01 à 06) via `nbconvert`. Il propose plusieurs modes d'exécution :

- **Exécution complète** : lance les notebooks 01 à 06 en séquence ;
- **Reprise partielle** : option `-from N` pour reprendre à partir d'un notebook spécifique ;
- **Notebook unique** : option `-only N` pour n'exécuter qu'une seule étape ;
- **Vérification uniquement** : option `-check-only` pour valider les imports sans lancer les calculs.

7.2.2 Garanties de reproductibilité

La reproductibilité est assurée par quatre mécanismes complémentaires :

1. **Graines aléatoires fixes** : `RANDOM_STATE = 42` dans tous les modules qui comportent une part d'aléatoire (`split`, `SMOTE`, forêt aléatoire, `dropout`).
2. **Environnement virtuel** : toutes les dépendances sont centralisées dans `requirements.txt` avec des contraintes de versions, à installer dans un environnement Python isolé.
3. **Pipeline déterministe** : toutes les transformations (`scaler`, encodeurs) sont sérialisées via `joblib` dans `outputs/models/` pour être réappliquées à l'inférence sans recalcul.
4. **Séparation stricte des données** : les ensembles de validation et test ne sont jamais utilisés pour l'apprentissage ni les transformations statistiques.

7.3 Tests unitaires

7.3.1 Structure de la suite de tests

Le dépôt contient une suite de tests unitaires structurée sous `tests/` et exécutable via `pytest -q` :

TABLE 7.2 – Suite de tests unitaires du projet

Fichier de test	Modules testés
tests/test_preprocessing.py	Pipeline de préparation des données
tests/test_models.py	Modèles ML et autoencodeur
tests/test_visualization.py	Génération de figures
tests/test_utils.py	Utilitaires et métriques
tests/test_ollama.py	Intégration LLM (Ollama)

Les fichiers de tests constituent un squelette structuré prêt à être alimenté. L’étape suivante consiste à implémenter les cas de test dans ces fichiers pour atteindre l’objectif de couverture à 80 %.

7.4 Perspectives d’amélioration

7.4.1 Améliorations techniques

Les résultats obtenus ouvrent plusieurs pistes d’amélioration algorithmique :

- **Modèles de gradient boosting** : intégration de LightGBM ou XGBoost, qui constituent l’état de l’art sur les données tabulaires et pourraient améliorer les performances supervisées tout en restant interprétables via SHAP.
- **Autoencodeur variationnel (VAE)** : passage d’un autoencodeur déterministe à un VAE qui modélise la distribution des données normales sous forme probabiliste — plus robuste pour détecter les anomalies de bord de distribution.
- **Détection en temps réel** : portage du pipeline vers une architecture streaming (Apache Kafka + Apache Flink) pour analyser les transactions au fil de l’eau, plutôt qu’en batch.
- **Apprentissage en ligne** : mise à jour continue des modèles au fil des nouvelles transactions validées par les auditeurs, pour s’adapter aux évolutions des comportements frauduleux.

7.4.2 Améliorations métier

Sur le plan fonctionnel, plusieurs extensions rendraient le système directement opérationnel en production :

- **Interface utilisateur** : tableau de bord (dashboard) pour les analystes permettant de consulter les alertes, de les valider ou de les rejeter, et de consulter les explications en langage naturel sans accéder au code.
- **Boucle de rétroaction (*feedback loop*)** : intégration des décisions humaines (fraude confirmée ou infirmée) pour ré-entraîner périodiquement les modèles et améliorer leur précision sur les données métier réelles.
- **Détection comportementale** : profiling par utilisateur ou par compte pour détecter non plus des transactions isolées anormales, mais des dérives comportementales dans le temps (ex : montants progressivement croissants, changements d’habitudes de transfert).

Conclusion

Ce chapitre a présenté l’architecture de déploiement et les mécanismes de reproductibilité du projet. La **structure modulaire** et l’orchestration via `run_all.py` fournissent une base maintenable et facilement reprise par un nouveau développeur. Les quatre garanties de reproductibilité — graines fixes, environnement versionné, artefacts sérialisés, séparation étanche des données — assurent que les résultats présentés dans ce rapport sont intégralement reproductibles. La suite de tests structurée pose les bases d’un contrôle qualité progressif. Les perspectives identifiées offrent de nombreuses pistes pour prolonger ce travail vers un **système de production opérationnel**, capable d’analyser des flux transactionnels en temps réel et d’apprendre de façon continue des retours des auditeurs.

Conclusion générale

Ce projet de fin d'études a abouti à un résultat concret : une **chaîne complète de détection intelligente d'anomalies financières**, opérationnelle, reproductible et explicable, développée selon la méthodologie CRISP-DM sur des données transactionnelles réelles issues d'une mission d'audit IT chez PwC Tunisie.

Récapitulatif de la démarche. Partant d'un corpus de 200 000 transactions financières, le travail a progressé de manière structurée : compréhension des enjeux métier, analyse exploratoire approfondie, préparation rigoureuse des données (14 variables, séparation étanche des partitions, transformation logarithmique, one-hot encoding), entraînement de cinq configurations de modèles, évaluation comparative et mise en place d'une double couche d'explicabilité (SHAP/LIME + Ollama).

Réponse à la problématique. Le pipeline construit répond positivement à la problématique initiale : il est **reproductible** (orchestration via `run_all.py`, graines fixes, versions épinglées), **efficace** (la forêt aléatoire avec SMOTE atteint un F1-score de $\sim 0,80$ et un PR-AUC de $\sim 0,84$ sur le test, contre $F1 = 0,0077$ pour la baseline), et **interprétable** (chaque transaction suspecte est accompagnée d'une explication en langage naturel via Ollama).

Difficultés et apports techniques. Trois défis ont retenu l'essentiel de mon attention : la gestion du déséquilibre extrême (ratio 1 : 774), la discipline anti-leakage à maintenir tout au long du prétraitement, et la sensibilité du seuil de détection de l'autoencoder. Sur le plan personnel, ce projet m'a permis de mettre en œuvre, sur un cas réel et contraint, un spectre technique inhabituel pour un projet de fin d'études : machine learning supervisé, deep learning, méthodes d'interprétabilité et intégration d'un LLM local — le tout dans un cadre méthodologique rigoureux imposé par le contexte professionnel.

Perspectives. Ce travail ouvre plusieurs axes de prolongement : l'intégration de modèles de gradient boosting (LightGBM, XGBoost), le passage à un autoencoder variationnel, le déploiement en temps réel sur un flux de transactions, et l'enrichissement de la couche d'explicabilité par des interfaces utilisateur dédiées aux analystes métier.

Bibliographie / Netographie

1. **Lopez-Rojas, E. A., Elmir, A., & Axelsson, S.** (2016). PaySim : A financial mobile money simulator for fraud detection. *The 28th European Modeling and Simulation Symposium (EMSS)*, Larnaca, Cyprus.
2. **Chapman, P. et al.** (2000). CRISP-DM 1.0 : Step-by-step data mining guide. SPSS Inc.
3. **Chawla, N. V. et al.** (2002). SMOTE : Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
4. **Breiman, L.** (2001). Random forests. *Machine Learning*, 45(1), 5–32.
5. **Hinton, G. E., & Salakhutdinov, R. R.** (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507.
6. **Lundberg, S. M., & Lee, S.-I.** (2017). A unified approach to interpreting model predictions. *NeurIPS*, 30.
7. **Ribeiro, M. T., Singh, S., & Guestrin, C.** (2016). Why should I trust you ? Explaining the predictions of any classifier. *KDD 2016*, pp. 1135–1144.
8. **Pedregosa, F. et al.** (2011). Scikit-learn : Machine Learning in Python. *JMLR*, 12, 2825–2830. <https://scikit-learn.org/stable/>
9. **Lemaître, G., Nogueira, F., & Aridas, C. K.** (2017). Imbalanced-learn. *JMLR*, 18(17), 1–5. <https://imbalanced-learn.org/stable/>
10. **Lundberg, S. M.** Documentation officielle SHAP. <https://shap.readthedocs.io/>
11. **Ribeiro, M. T.** Documentation officielle LIME. <https://lime-ml.readthedocs.io/>
12. **Ollama Inc.** Documentation officielle Ollama. <https://ollama.com/>
13. **Chaieb, Y.** *Intelligent Anomaly Detection* — Dépôt GitHub du projet. PwC Tunisie / ES-PRIT, 2026. <https://github.com/yousra0/Intelligent-Anomaly-Detection>
14. **Auteur non spécifié.** La méthodologie CRISP-DM : présentation des six phases. Le1817.com. <https://www.le1817.com/blog/la-methodologie-crisp-dm>

A Hyperparamètres détaillés des modèles

A.1 Régression logistique

```
1 LR_DEFAULTS = {
2     "C": 0.1,
3     "max_iter": 1000,
4     "solver": "lbfgs",
5     "class_weight": "balanced",
6     "random_state": 42,
7     "n_jobs": -1,
8 }
```

Listing A.1 – Hyperparamètres LR (src/models/ml_models.py)

A.2 Forêt aléatoire

```
1 RF_DEFAULTS = {
2     "n_estimators": 300,
3     "max_depth": 10,
4     "min_samples_leaf": 5,
5     "class_weight": "balanced",
6     "random_state": 42,
7     "n_jobs": -1,
8 }
```

Listing A.2 – Hyperparamètres RF (src/models/ml_models.py)

A.3 Autoencoder

```
1 AE_DEFAULTS = {
2     "encoder_dims": [10, 7],
3     "bottleneck_dim": 4,
4     "decoder_dims": [7, 10],
5     "activation": "relu",
6     "output_activation": "linear",
7     "dropout_rate": 0.2,
8     "use_batch_norm": True,
9     "l2_reg": 1e-5,
10    "epochs": 100,
11    "batch_size": 256,
12    "learning_rate": 1e-3,
13    "patience": 10,
14    "val_split": 0.1,
```

```
15     "threshold_percentile": 95,  
16 }
```

Listing A.3 – Hyperparamètres AE (src/models/autoencoder.py)

B Pipeline de préparation — variables

B.1 Variables normalisées (StandardScaler)

```
1 SCALE_COLS = [  
2     "step", "hour", "day", "week",  
3     "log_amount", "balance_diff_orig"  
4 ]
```

B.2 Variables binaires (pas de normalisation)

```
1 BINARY_COLS = [  
2     "high_risk_hour",  
3     "is_transfer_or_cashout",  
4     "dest_zero_balance"  
5 ]
```

B.3 Variables one-hot (type de transaction)

```
1 TYPE_COLS = [  
2     "type_CASH_IN", "type_CASH_OUT", "type_DEBIT",  
3     "type_PAYMENT", "type_TRANSFER"  
4 ]
```

C Planning de réalisation

TABLE C.1 – Planning de réalisation du projet de fin d'études

Phase CRISP-DM	Activités principales	Durée
Compréhension métier	Analyse des besoins, définition des objectifs	Semaine 1
Compréhension données	EDA, notebook 01, analyse qualité	Semaines 2–3
Préparation données	Feature engineering, split, normalisation	Semaines 3–4
Modélisation	LR, RF, Autoencoder, réglage seuils	Semaines 5–7
Évaluation	Métriques, SHAP, LIME, Ollama	Semaines 7–9
Déploiement	Pipeline, tests, documentation	Semaines 9–10
Rédaction du rapport	Rédaction, révision, mise en forme	Semaines 8–12

D Exemple d'explication générée par Ollama

```
1 Transaction ID : TXN_00123
2 Montant : 245 000 | Type : TRANSFER | Heure : 03h00
3
4 Cette transaction est signalee comme suspecte pour les raisons
   suivantes :
5 1. Type TRANSFER : seul type associe a des fraudes dans le corpus
   d'audit.
6 2. Heure 3h00 : heure a risque eleve (ratio fraude 10x superieur a
   la
7   moyenne horaire).
8 3. Difference de solde emetteur (balance_diff_orig = 245 000)
   anormalement
9   elevee, coherente avec un virement frauduleux.
10 4. Score de reconstruction AE (MSE = 1.87) superieur au seuil (
   theta = 1.67)
11   : pattern inhabituel non reconnu par le modele.
12
13 Recommandation : Soumettre a une verification manuelle par un
   auditeur.
```

Listing D.1 – Exemple d'explication pour une transaction suspecte